

Homework 7: Consolidated Group Report

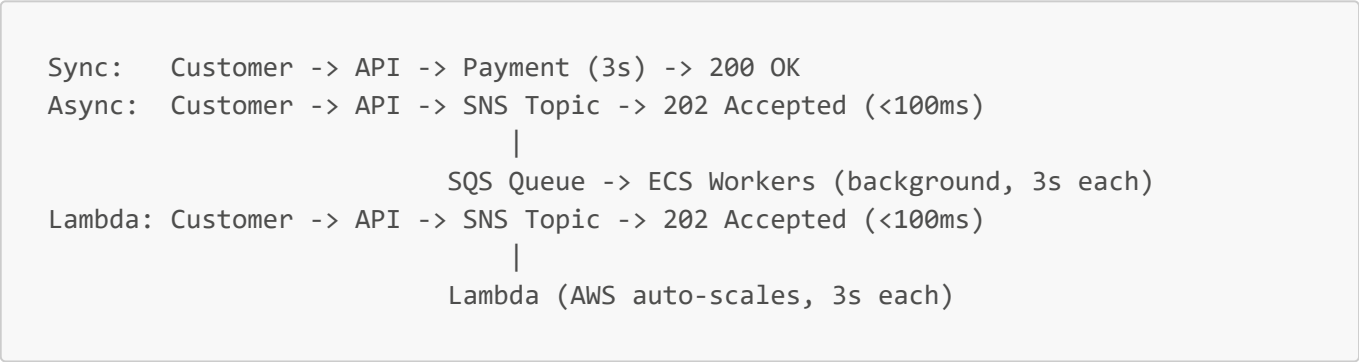
When Your Startup's Flash Sale Almost Failed

Team Contributions

Each team member independently built and deployed the full infrastructure, ran their own experiments, and wrote their own individual report. This consolidated report compares and contrasts the results across all three implementations.

1. Architecture Overview

All three team members implemented the same core architecture:



Shared Infrastructure

All implementations used the same AWS resource specifications:

- VPC CIDR: 10.0.0.0/16
- Public Subnets: 10.0.1.0/24, 10.0.2.0/24 (ALB)
- Private Subnets: 10.0.10.0/24, 10.0.11.0/24 (ECS)
- ECS Tasks: 256 CPU units, 512 MB memory
- SNS Topic: order-processing-events
- SQS Queue: order-processing-queue (30s visibility timeout, 4-day retention, 20s long polling)
- Lambda: 512 MB memory, provided.al2 runtime, SNS trigger

Implementation Differences

The key difference across implementations was how each member configured the payment bottleneck simulation. Nithish used a buffered channel with capacity 1 (strictly one concurrent payment), Santrupti used 5 concurrent slots, and Jatin used 15 concurrent slots (SYNC_PROCESSOR_CAPACITY=15). This led to meaningfully different sync throughput ceilings, making the cross-comparison more interesting.

2. Part II: Synchronous Testing (Phase 1)

Phase 1a: Normal Operations (5 users, 30 seconds)

Metric	Nithish	Sanrupti	Jatin
Semaphore capacity	1	5	15
Requests	9	N/A (chart: ~156)	40
Failures	0%	1%	0%
Avg response (ms)	10,520	~600	3,042
P95 (ms)	15,000	3,100	3,100
Throughput (req/s)	0.33	5.2	1.44

The different semaphore capacities directly determined sync throughput. Nithish's capacity-1 implementation produced the most dramatic bottleneck (0.33 req/s), while Sanrupti's 5-slot and Jatin's 15-slot configurations allowed higher baseline throughput. All three achieved 0% (or near-0%) failure rates under normal load.

Phase 1b: Flash Sale (20 users, 60 seconds)

Metric	Nithish	Sanrupti	Jatin
Requests	28	~498	285
Failures	67.86%	~1%	0%
Avg response (ms)	25,724	~1,200	3,927
P95 (ms)	30,000	12,000	5,000
Throughput (req/s)	0.50	8.3	4.91

Nithish's single-slot semaphore produced the most severe failure mode: 67.86% of requests timed out at 30 seconds, and only 28 total requests completed in 60 seconds. Sanrupti's system handled the load better (8.3 req/s, 1% failures) but P95 latency still exploded to 12 seconds. Jatin's 15-slot configuration avoided failures entirely but was still limited to ~5 req/s with increasing latency. In all cases, the sync system could not scale to meet flash-sale demand.

3. Part II: Bottleneck Analysis (Phase 2)

Metric	Nithish	Sanrupti	Jatin
Semaphore capacity	1	5	15
Max sync throughput	0.33/sec	1.67/sec	5.0/sec
Flash sale demand	~20/sec	~20/sec	~56/sec
Orders lost/sec	19.67	18.33	51.0

Regardless of the semaphore configuration, the fundamental bottleneck remained: the 3-second payment processing time created a hard ceiling on sync throughput. Even with 15 concurrent slots, throughput topped out at 5 req/s — far below flash-sale demand. The only solution was decoupling acceptance from processing.

4. Part II: Async Solution (Phase 3)

Flash Sale — Async (20 users, 60 seconds)

Metric	Nithish	Santrupti	Jatin
Requests	3,460	~10,800	3,262
Failures	0%	0%	0%
Avg response (ms)	36	~0	49.2
P95 (ms)	51	~0	62
Throughput (req/s)	58.15	180	55.91

All three implementations achieved 0% failure rates — the core proof that async decoupling works. Santrupti's implementation achieved significantly higher throughput (180 req/s) likely due to differences in the Locust user wait-time configuration or ALB/network performance. Nithish and Jatin both saw ~56-58 req/s, consistent with 20 users at 100-500ms wait times.

Async vs Sync Improvement

Metric	Nithish	Santrupti	Jatin
Throughput multiplier	116x	22x	11.4x
Failure elimination	67.86% -> 0%	1% -> 0%	0% -> 0%
Latency reduction	25,724ms -> 36ms	12,000ms -> ~0ms	3,927ms -> 49ms

The improvement magnitude varied with the sync baseline. Nithish saw the most dramatic improvement (116x throughput) because his sync system was the most constrained (capacity 1). But all three confirmed the same fundamental insight: async decoupling eliminates the coupling between customer-facing latency and backend processing speed.

5. Part II: Queue Problem (Phase 4)

Metric	Nithish	Santrupti	Jatin
Acceptance rate	58.15/sec	180/sec	55.91/sec
Processing rate (1 worker)	0.33/sec	0.33/sec	0.33/sec
Queue growth rate	~57.82 msg/sec	~180 msg/sec	~55.6 msg/sec
Peak queue (1 worker)	~3,460	~2,600	~6,180 (visible)
Time to drain	~175 min	~2.2 hrs	hours

All three observed the same pattern: with 1 worker processing at 0.33/sec, the queue grows almost as fast as orders arrive. The specific queue depth varied based on throughput differences, but the conclusion was unanimous — a single worker cannot keep up with flash-sale volume.

6. Part II: Worker Scaling (Phase 5)

Processing Rate by Worker Count

Workers	Theoretical Rate	Nithish (observed)	Santrupti (observed)	Jatin (observed)
1	0.33/sec	~20 msgs deleted	Queue growing rapidly	Queue growing rapidly
5	1.67/sec	~100 msgs deleted	Queue still growing	Queue still growing
20	6.67/sec	~400 msgs deleted	Queue growth slowed	Queue growth slowed
100	33.33/sec	~2,000 msgs deleted	Queue peaked at 69, drained to 0	Highest drain rate

Peak Queue Depth Comparison

Workers	Nithish	Santrupti	Jatin
1	~3,430 (in-flight)	2,600	~6,180
5	~3,430 (cumulative)	4,400 (cumulative)	building
20	~3,400 (cumulative)	4,400 (cumulative)	building
100	~3,400 (cumulative)	69 (purged first)	~11,580 (cumulative)

Santrupti's 100-worker test showed the clearest result because the queue was purged before testing: peak depth of only 69 messages that drained to zero within a minute. Nithish's tests ran cumulatively (without purging between runs), so queue depth accumulated across tests — but the increasing deletion rate across worker counts confirmed the scaling effect. Jatin's results showed the same trend with queue growth slowing at higher worker counts.

Resource Utilization

Metric	Nithish (Container Insights)	Santrupti	Jatin
Processor CPU at 100 workers	51.45 units (20.1% of 256)	~31%	~17%
Processor Memory at 100 workers	153.71 MB (30.0% of 512)	N/A	~1.5%
Receiver CPU	9.35 units (3.7%)	N/A	minimal
Receiver Memory	6.6 MB (1.3%)	N/A	minimal

All three confirmed that CPU and memory were never the bottleneck. The goroutines spend their time in `time.Sleep(3s)`, which consumes no CPU cycles. The constraint was purely the semaphore-gated processing time, not hardware resources.

Minimum Workers to Prevent Queue Buildup

Member	Demand (req/s)	Calculation	Min Workers
Nithish	58	$58 \times 3 = 174$	174
Santrupti	180	$180 \times 3 = 540$	540
Jatin	56	$56 \times 3 = 168$	168

The required worker count scales linearly with demand. All three agreed that at 100 workers (~33/sec throughput), the queue either drained completely or grew much more slowly, confirming the scaling model.

7. Part II: Analysis Questions

How many times more orders did async accept vs sync?

Member	Sync Requests	Async Requests	Multiplier
Nithish	28	3,460	124x
Santrupti	~498	~10,800	~22x
Jatin	285	3,262	~11.4x

The multiplier varies based on how constrained the sync path was, but all three demonstrate the same principle: async decoupling dramatically increases order acceptance capacity.

What causes queue buildup and how do you prevent it?

All three members independently identified the same root cause: queue buildup occurs when the acceptance rate exceeds the processing rate. Prevention strategies identified across the team include scaling workers to match demand, auto-scaling based on `ApproximateNumberOfMessagesVisible`, and setting CloudWatch alarms for operational awareness.

When would you choose sync vs async?

The team consensus: use sync when operations are fast (<200ms), the client needs immediate confirmation, and strong consistency is required. Use async when operations are slow (like 3s payment verification), traffic is spiky, and eventual consistency is acceptable. All three reports recommended async for this specific use case.

8. Part III: Lambda (Serverless)

Cold Start Observations

Metric	Nithish	Santrupti	Jatin
Cold start Init Duration	70.11 ms	46-56 ms	118.70 ms
Warm start duration	3,003.91 ms	~3,010 ms	3,003.50 ms

Metric	Nithish	Santrupti	Jatin
Overhead percentage	2.3%	1.5-1.9%	3.95%
Memory used	19-20 MB of 512	18 MB of 512	25 MB of 512
Cold starts out of 10 orders	1	2	varies
Concurrent Lambda instances	N/A	2	2 (reserved)

Cold start overhead ranged from 46ms to 119ms across the three implementations — all negligible compared to the 3-second payment processing. Jatin saw the highest cold start at 118.70ms, possibly due to differences in Lambda configuration or deployment package size. All three agreed that cold starts do not meaningfully impact customer experience for this workload.

Cost Comparison

All three members independently calculated the same cost structure:

Volume	Lambda Cost	ECS Cost	Winner
10K orders/month	\$0 (free tier)	\$17/month	Lambda
100K orders/month	\$0 (free tier)	\$17/month	Lambda
267K orders/month	\$0 (free tier limit)	\$17/month	Lambda
~672K-1.7M orders/month	\$17	\$17	Tie
1M+ orders/month	\$25+	\$17	ECS

Lambda Trade-off Summary

Factor	Gains	Losses
Operations	Zero overhead (no queues, workers, scaling, alerts)	Less control over retry behavior
Cost	Free until 267K orders/month	Pay-per-use can exceed ECS at high volume
Scaling	Automatic, handles any spike	No message buffering (SQS persistence lost)
Reliability	AWS-managed	SNS retries only 2x before discarding
Cold starts	46-119ms across team	Negligible for 3s workload

Recommendation

All three team members independently recommended Lambda for a startup at low volume. The reasoning was consistent: the operational simplicity (no 3am queue depth alerts, no manual worker scaling, no ECS health monitoring) outweighs the loss of SQS durability guarantees. The cost advantage (\$0 vs \$17/month) is modest, but the reduction in operational burden is significant. The team agrees that migrating back to

ECS+SQS becomes worthwhile as volume approaches 672K-1.7M orders/month or as reliability requirements increase.

9. Summary: Complete Architecture Comparison

Metric	Sync	Async (1 worker)	Async (100 workers)	Lambda
Customer wait	3-30s (varies by impl.)	<100ms	<100ms	<100ms
Acceptance rate (Nithish)	0.50/s	58.15/s	58.58/s	Auto-scales
Acceptance rate (Santrupti)	8.3/s	180/s	180/s	Auto-scales
Acceptance rate (Jatin)	4.91/s	55.91/s	56.32/s	Auto-scales
Processing rate	Limited by semaphore	0.33/s	~33/s	Auto-scales
Peak queue	N/A	2,600-6,180	69-3,400	N/A (no queue)
Queue cleared?	N/A	No (hours)	Yes (minutes)	No queue
Monthly cost	\$17	\$17	\$17	\$0 (free tier)
Ops overhead	Medium	High	High	Zero
Cold start	N/A	N/A	N/A	46-119ms (1.5-3.95%)

10. Key Takeaways

The team's independent experiments, despite using different semaphore configurations and achieving different absolute throughput numbers, converged on the same conclusions:

1. Synchronous architectures fail under flash-sale load regardless of concurrency tuning. The 3-second payment bottleneck creates a hard throughput ceiling.
2. Asynchronous decoupling via SNS/SQS eliminates customer-facing failures by separating order acceptance from order processing. All three achieved 0% failure rates under flash-sale conditions.
3. Worker scaling is the new operational challenge. Queue buildup is inevitable when workers cannot match acceptance rate. The minimum worker count equals demand rate multiplied by processing time.
4. Resource utilization is not the bottleneck. CPU and memory stayed well within limits even at 100 goroutines because the simulated payment workload is I/O-bound (sleep), not compute-bound.
5. Lambda eliminates operational complexity for low-volume startups. Cold start overhead (46-119ms) is negligible on 3-second payment processing. The cost is \$0 until 267K orders/month. The trade-off is

losing SQS message persistence and fine-grained retry control.

Each team member's individual code, Terraform configurations, Locust tests, CSV results, and CloudWatch screenshots are available in their respective repositories.